1 Assignment Overview

What you will implement

1. Byte-pair encoding (BPE) tokenizer DONE
2. Transformer language model (LM)
3. The cross-entropy loss function and the AdamW optimizer
4. The training loop, with support for serializing and loading model and optimizer state

What you will run

1. Train a BPE tokenizer on the TinyStories dataset. DONE
2. Run your trained tokenizer on the dataset to convert it into a sequence of integer IDs.
3. Train a Transformer LM on the TinyStories dataset.
4. Generate samples and evaluate perplexity using the trained Transformer LM.
5. Train models on OpenWebText and submit your attained perplexities to a leaderboard.

2 Byte-Pair Encoding (BPE) Tokenizer

2.1 The Unicode Standard

In Python, you can use the `ord()` function to convert a single Unicode character into its integer representation. The `chr()` function converts an integer Unicode code point into a string with the corresponding character.

Problem (unicode1): Understanding Unicode (1 point)

(a) What Unicode character does `chr(0)` return?
Answer: `'\x00'`

(b) How does this character's string representation `__repr__()` differ from its printed representation?
Answer: The `__repr__()` is designed to be **unambiguous** and is mainly for developers, showing the technical details of the object. On the other hand, the printed representation is meant to be **readable** and user-friendly.

(c) What happens when this character occurs in text? It may be helpful to play around with the following in your Python interpreter and see if it matches your expectations:

Answer: Code below.

```
>>> chr(0)
'\x00'
>>> print(chr(0))

>>> "this is a test" + chr(0) + "string"
'this is a test\x00string'
>>> print("this is a test" + chr(0) + "string")
this is a teststring
```

2.2 Unicode Encodings

- UTF-8: The Efficient Variable-Length Encoder, It uses 1 to 4 bytes per character.
- UTF-16: The Middle Ground, It uses either 2 or 4 bytes per character.
- UTF-32: The Simple Fixed-Length Encoder, It uses exactly 4 bytes for every character, regardless of what it is.

Problem (unicode2): Unicode Encodings (3 point)

(a) What are some reasons to prefer training our tokenizer on UTF-8 encoded bytes, rather than UTF-16 or UTF-32? It may be helpful to compare the output of these encodings for various input strings.
Answer: `'\x00'`

(b) Consider the following (incorrect) function, which is intended to decode a UTF-8 byte string into a Unicode string. Why is this function incorrect? Provide an example of an input byte string that yields incorrect results.

```
def decode_utf8_bytes_to_str_wrong(bytestring: bytes):
    return "".join([bytes([b]).decode("utf-8") for b in bytestring])

>>> decode_utf8_bytes_to_str_wrong("hello".encode("utf-8"))
'hello'
```

Example: `b'\xe4\xb8\xad'`.
Answer: This function is incorrect because it attempts to decode each byte individually, which fails for any character encoded as a multi-byte sequence, resulting in a `UnicodeDecodeError`.

(c) Give a two byte sequence that does not decode to any Unicode character(s).
Example: `b'\xff\xff'`.
Answer: This sequence is invalid because the byte `0xFF` is explicitly forbidden in the UTF-8 standard and does not correspond to any valid lead or continuation byte in the encoding scheme.

2.3 Subword Tokenization

Sennrich et al. [2016] propose to use byte-pair encoding (BPE; Gage, 1994), a compression algorithm that iteratively replaces ("merges") the most frequent pair of bytes with a single, new unused index.

2.4 BPE Tokenizer Training

Vocabulary initialization

Pre-tokenization Use regular expressions to split the sentence into independent punctuation marks and independent items that may include spaces, and improve the merging effect by counting the occurrence frequency of these independent items.

```
>>> PAT = r"""'(?:[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+| ?[^\s\p{L}\p{N}]+|\s+(?!\S)|\s+"""
>>> # requires `regex` package
>>> import regex as re
>>> re.findall(PAT, "some text that i'll pre-tokenize")
['some', ' text', ' that', ' i', "'ll", ' pre', '-', 'tokenize']
```

Compute BPE merges During the BPE merging process, two **principles** must be followed:

- Do not **cross** pre-tokenization boundaries (this is already guaranteed algorithmically);
- Handle issues with a **reproducible conflict resolution strategy**, such as selecting the lexicographically largest word pair to merge when the occurrence counts are the same.

Special tokens

2.5 Experimenting with BPE Tokenizer Training

Parallelizing pre-tokenization Using the provided segmentation code and the `multiprocessing` library, parallelize the tokenization statistics.

Removing special tokens before pre-tokenization

Optimizing the merging step After performing a statistical pass to obtain a tokenization table with frequencies, perform merging only within that tokenization table.

Low-Resource/Downscaling Tip: Profiling

You should use profiling tools like `cProfile` or `scalene` to identify the bottlenecks in your implementation, and focus on optimizing those.

Problem (train_bpe): BPE Tokenizer Training (15 points)

Deliverable: Write a function that, given a path to an input text file, trains a (byte-level) BPE tokenizer. Your BPE training function should handle (at least) the following input parameters:

`input_path`: `str` Path to a text file with BPE tokenizer training data.

`vocab_size`: `int` A positive integer that defines the maximum final vocabulary size (including the initial byte vocabulary, vocabulary items produced from merging, and any special tokens).

`special_tokens`: `list[str]` A list of strings to add to the vocabulary. These special tokens do not otherwise affect BPE training.

Your BPE training function should return the resulting vocabulary and merges:

`vocab`: `dict[int, bytes]` The tokenizer vocabulary, a mapping from `int` (token ID in the vocabulary) to `bytes` (token bytes).

`merges`: `list[tuple[bytes, bytes]]` A list of BPE merges produced from training. Each list item is a `tuple` of `bytes` (`<token1>`, `<token2>`), representing that `<token1>` was merged with `<token2>`. The merges should be ordered by order of creation.

Insights: BPE Tokenizer Training

Python plus Rust: Python is used for rapid prototyping, while Rust handles the actual deployment of the product.

Glitch Tokens: Due to insufficiently strict data cleaning, some meaningless words—namely **Mojibake**—may appear during the BPE training process. Moreover, because of BPE's **greedy, unsupervised, and non-semantic nature**, a large number of such meaningless word merges occur, turning them into abnormally long and nonsensical words.

Vocabulary Bloat (Dead Tokens): Because BPE is incremental and never removes merges, the intermediate products of long words permanently occupy the vocabulary, leading to wasted model parameters and tokenization fragmentation. In industry, there are three approaches to address this issue: First, absolutely aggressive data cleaning; second, stricter regex filtering. Both of these are common methods for limiting glitch tokens. The third approach is to switch to the **Unigram** algorithm, which is frequently used by Google-family models (such as T5 and some Llama variants).

Unigram: Unigram works by "subtraction": it first initializes an extremely large vocabulary (e.g., 100,000 words) and **then calculates how much the overall corpus compression rate would be affected if a given word were removed**. Those useless intermediate composite words are directly eliminated and discarded during training.

Problem (train_bpe_tinystories): BPE Training on TinyStories (2 points)

(a) Train a byte-level BPE tokenizer on the TinyStories dataset, using a maximum vocabulary size of 10,000. Make sure to add the TinyStories `<|endoftext|>` special token to the vocabulary. Serialize the resulting vocabulary and merges to disk for further inspection. How many hours and memory did training take? What is the longest token in the vocabulary? Does it make sense?

Resource requirements: $\leq$ 30 minutes (no GPUs), $\leq$ 30GB RAM
Resource used: 10.78 seconds, 2.6GB RAM
Longest token: ' disappointment'

(b) Profile your code. What part of the tokenizer training process takes the most time?

Answer: `process_chunk_with_special_tokens` takes up the vast majority of the time.

Problem (train_bpe_expts_owt): BPE Training on OpenWebText (2 points)

(a) Train a byte-level BPE tokenizer on the OpenWebText dataset, using a maximum vocabulary size of 32,000. Serialize the resulting vocabulary and merges to disk for further inspection. What is the longest token in the vocabulary? Does it make sense?

Resource requirements: $\leq$ 12 hours (no GPUs), $\leq$ 100GB RAM
Resource used: 6 minutes, 7.0GB RAM
Longest token: '————————————————————————'

(b) Compare and contrast the tokenizer that you get training on TinyStories versus OpenWebText.

Answer: Because TinyStories has an extremely limited vocabulary, a 10k vocab size forces the BPE tokenizer to over-merge, creating tokens that represent entire multi-word phrases. In contrast, OpenWebText's diverse internet text requires the BPE algorithm to rely heavily on subwords (like prefixes and suffixes) to represent its massive vocabulary within a 32k limit, resulting in shorter, more fragmented tokens for rare words.

2.6 BPE Tokenizer: Encoding and Decoding

2.6.1 Encoding text

Special tokens.

Memory considerations.

2.6.2 Decoding text

Problem (tokenizer): Implementing the tokenizer (15 points)

Deliverable: Implement a `Tokenizer` class that, given a vocabulary and a list of merges, encodes text into integer IDs and decodes integer IDs into text. Your tokenizer should also support user-provided special tokens (appending them to the vocabulary if they aren't already there).

Answer: code in appendix.

2.7 Experiments

Problem (tokenizer_experiments): Experiments with tokenizers (4 points)

(a) Sample 10 documents from TinyStories and OpenWebText. Using your previously-trained TinyStories and OpenWebText tokenizers (10K and 32K vocabulary size, respectively), encode these sampled documents into integer IDs. What is each tokenizer's compression ratio (bytes/token)?

Answer: The TinyStories tokenizer achieves a higher compression ratio (5.5 bytes/-token) because its vocabulary easily memorizes long, frequent whole words from its restricted domain. In contrast, the OpenWebText tokenizer has a lower compression ratio (3.8 bytes/token) as it must rely heavily on shorter subwords to encode a highly diverse vocabulary.

(b) What happens if you tokenize your OpenWebText sample with the TinyStories tokenizer? Compare the compression ratio and/or qualitatively describe what happens.

Answer: Tokenizing OpenWebText with the TinyStories tokenizer causes a severe domain mismatch, forcing the tokenizer to aggressively fragment unfamiliar web text into individual characters or very small byte sequences. Consequently, the token count explodes, resulting in a dramatically lower (worse) compression ratio compared to using the native OpenWebText tokenizer.

(c) Estimate the throughput of your tokenizer (e.g., in bytes/second). How long would it take to tokenize the Pile dataset (825GB of text)?

Answer: The throughput of the tokenizer depends on the implementation details, but a rough estimate for a modern CPU might be around 100 MB/second. Given the Pile dataset's size of 825GB, it would take approximately 8.25 hours to tokenize the entire dataset.

(d) Using your TinyStories and OpenWebText tokenizers, encode the respective training and development datasets into a sequence of integer token IDs. We'll use this later to train our language model. We recommend serializing the token IDs as a NumPy array of datatype `uint16`. Why is `uint16` an appropriate choice?

Answer: `uint16` is an appropriate choice because it can represent all possible token IDs within the range of 0 to 65,535, which is sufficient for most practical applications and allows for efficient memory usage.

3 Transformer Language Model Architecture

3.1 Transformer LM

3.1.1 Token Embeddings

3.1.2 Pre-norm Transformer Block

3.2 Output Normalization and Embedding

3.3 Remark: Batching, Einsum and Efficient Computation

3.3.1 Mathematical Notation and Memory Ordering

3.4 Basic Building Blocks: Linear and Embedding Modules

3.4.1 Parameter Initialization

3.4.2 Linear Module

Problem (linear): Implementing the linear module **(1 point)**

Deliverable: Implement a `Linear` class that inherits from `torch.nn.Module` and performs a linear transformation. Your implementation should follow the interface of PyTorch's built-in `nn.Linear` module, except for not having a `bias` argument or parameter. We recommend the following interface:

`def __init__(self, in_features, out_features, device=None, dtype=None)` Construct a linear transformation module. This function should accept the following parameters:

`in_features`: `int` final dimension of the input

`out_features`: `int` final dimension of the output

`device`: `torch.device | None = None` Device to store the parameters on

`dtype`: `torch.dtype | None = None` Data type of the parameters

`def forward(self, x: torch.Tensor) -> torch.Tensor` Apply the linear transformation to the input.

Make sure to:

- subclass `nn.Module`
- call the superclass constructor
- construct and store your parameter as W (not $W^\top$) for memory ordering reasons, putting it in an `nn.Parameter`
- of course, don't use `nn.Linear` or `nn.functional.linear`

Code below;

3.4.3 Embedding Module

Problem (embedding): Implement the embedding module **(1 point)**

Deliverable: Implement the `Embedding` class that inherits from `torch.nn.Module` and performs an embedding lookup. Your implementation should follow the interface of PyTorch's built-in `nn.Embedding` module. We recommend the following interface:

```
def __init__(self, num_embeddings, embedding_dim, device=None, dtype=None)
```

Construct an embedding module. This function should accept the following parameters:

`num_embeddings`: `int` Size of the vocabulary

`embedding_dim`: `int` Dimension of the embedding vectors, i.e., d_{model}

`device`: `torch.device | None = None` Device to store the parameters on

`dtype`: `torch.dtype | None = None` Data type of the parameters

```
def forward(self, token_ids: torch.Tensor) -> torch.Tensor
```

Lookup the embedding vectors for the given token IDs.

Make sure to:

- subclass `nn.Module`
- call the superclass constructor
- initialize your embedding matrix as a `nn.Parameter`
- store the embedding matrix with the `d_model` being the final dimension
- of course, don't use `nn.Embedding` or `nn.functional.embedding`

Again, use the settings from above for initialization, and use `torch.nn.init.trunc_normal_` to initialize the weights.
To test your implementation, implement the test adapter at `[adapters.run_embedding]`. Then, run `uv run pytest -k test_embedding`.

3.5 Pre-Norm Transformer Block

3.5.1 Root Mean Square Layer Normalization

Problem (rmsnorm): Root Mean Square Layer Normalization (1 point)

Deliverable: Implement RMSNorm as a `torch.nn.Module`. We recommend the following interface:

```
def __init__(self, d_model: int, eps: float = 1e-5, device=None, dtype=None)
```

Construct the RMSNorm module. This function should accept the following parameters:

`d_model: int` Hidden dimension of the model

`eps: float = 1e-5` Epsilon value for numerical stability

`device: torch.device | None = None` Device to store the parameters on

`dtype: torch.dtype | None = None` Data type of the parameters

```
def forward(self, x: torch.Tensor) -> torch.Tensor
```

Process an input tensor of shape `(batch_size, sequence_length, d_model)` and return a tensor of the same shape.

Note: Remember to upcast your input to `torch.float32` before performing the normalization (and later downcast to the original dtype), as described above.
To test your implementation, implement the test adapter at `[adapters.run_rmsnorm]`. Then, run `uv run pytest -k test_rmsnorm`.

3.5.2 Position-Wise Feed-Forward Network

Problem (positionwise_feedforward): Implement the position-wise feed-forward network (2 points)

Deliverable: Implement the SwiGLU feed-forward network, composed of a SiLU activation function and a GLU.

Note: in this particular case, you should feel free to use `torch.sigmoid` in your implementation for numerical stability.

You should set d_{ff} to approximately $\frac{8}{3} \times d_{\text{model}}$ in your implementation, while ensuring that the dimensionality of the inner feed-forward layer is a multiple of 64 to make good use of your hardware. To test your implementation against our provided tests, you will need to implement the test adapter at `[adapters.run_swiglu]`. Then, run `uv run pytest -k test_swiglu` to test your implementation.

A Complete Source Code

A.1 Fast BPE Implementation (Rust)

```rust
use pyo3::prelude::*;
use std::collections::{HashMap, HashSet, BinaryHeap};
use std::cmp::Ordering;

#[derive(Eq, PartialEq)]
struct HeapItem {
    count: usize,
    pair: (u32, u32),
}

impl Ord for HeapItem {
    fn cmp(&self, other: &Self) -> Ordering {
        self.count.cmp(&other.count)
            .then_with(|| self.pair.cmp(&other.pair))
    }
}

impl PartialOrd for HeapItem {
    fn partial_cmp(&self, other: &Self) -> Option<Ordering> {
        Some(self.cmp(other))
    }
}

#[pyfunction]
fn train_bpe_rust(
    global_counts: HashMap<Vec<u8>, usize>,
    target_vocab_size: usize,
) -> PyResult<(Vec<(Vec<u8>, Vec<u8>)>, HashMap<usize, Vec<u8>>)> {

    let mut vocab: HashMap<u32, Vec<u8>> = (0..256)
        .map(|i| (i as u32, vec![i as u8]))
        .collect();
    let mut next_id = 256u32;
    let num_merges = target_vocab_size.saturating_sub(256);
    let mut merges = Vec::with_capacity(num_merges);

    let mut words: Vec<Vec<u32>> = Vec::new();
    let mut word_freqs: Vec<usize> = Vec::new();

    for (bytes, freq) in global_counts {
        words.push(bytes.into_iter().map(|b| b as u32).collect());
        word_freqs.push(freq);
    }

    let mut pair_counts: HashMap<(u32, u32), usize> = HashMap::new();
    let mut pair_to_words: HashMap<(u32, u32), HashSet<usize>> = HashMap::new();

    for (idx, word_ids) in words.iter().enumerate() {
        let freq = word_freqs[idx];
        for pair in word_ids.windows(2) {
            let p = (pair[0], pair[1]);
            *pair_counts.entry(p).or_insert(0) += freq;
            pair_to_words.entry(p).or_insert_with(HashSet::new).insert(idx);
        }
```

```
    }

    let mut heap: BinaryHeap<HeapItem> = pair_counts.iter()
        .map(|(&pair, &count)| HeapItem { count, pair })
        .collect();

    for _ in 0..num_merges {
        let mut best_pair = None;
        while let Some(top) = heap.pop() {
            if let Some(&current_count) = pair_counts.get(&top.pair) {
                if current_count == top.count {
                    best_pair = Some(top.pair);
                    break;
                }
            }
        }

        let p = match best_pair {
            Some(p) => p,
            None => break,
        };

        let p1_bytes = vocab.get(&p.0).unwrap().clone();
        let p2_bytes = vocab.get(&p.1).unwrap().clone();
        let mut new_token_bytes = p1_bytes;
        new_token_bytes.extend(p2_bytes);

        merges.push((vocab.get(&p.0).unwrap().clone(), vocab.get(&p.1).unwrap().clone()));
        vocab.insert(next_id, new_token_bytes);

        let new_id = next_id;
        next_id += 1;

        if let Some(word_indices) = pair_to_words.remove(&p) {
            pair_counts.remove(&p);

            for &word_idx in &word_indices {
                let freq = word_freqs[word_idx];
                let old_ids = &words[word_idx];
                let mut new_ids = Vec::new();

                let mut i = 0;
                while i < old_ids.len() {
                    if i < old_ids.len() - 1 && old_ids[i] == p.0 && old_ids[i+1] == p.1 {
                        new_ids.push(new_id);
                        i += 2;
                    } else {
                        new_ids.push(old_ids[i]);
                        i += 1;
                    }
                }

                for pair in old_ids.windows(2) {
                    let old_p = (pair[0], pair[1]);
                    if let Some(count) = pair_counts.get_mut(&old_p) {
                        *count -= freq;
                    }
                }

```

```
                words[word_idx] = new_ids;
                let updated_ids = &words[word_idx];
                for pair in updated_ids.windows(2) {
                    let new_p = (pair[0], pair[1]);
                    let count = pair_counts.entry(new_p).or_insert(0);
                    *count += freq;
                    pair_to_words.entry(new_p).or_insert_with(HashSet::new).insert(word_idx);
                    heap.push(HeapItem { count: *count, pair: new_p });
                }
            }
        }
    }

    let final_vocab = vocab.into_iter().map(|(k, v)| (k as usize, v)).collect();
    Ok((merges, final_vocab))
}

#[pymodule]
fn fast_bpe(m: &Bound<'_, pyo3::types::PyModule>) -> PyResult<()> {
    m.add_function(wrap_pyfunction!(train_bpe_rust, m)?)?;
    Ok(())
}
```

A.2 Python BPE Trainer

```python
import os
import time
import pickle
import regex as re
import multiprocessing
from typing import BinaryIO
from collections import Counter

import fast_bpe

# ==========================================
# 1. 全局配置与正则定义
# ==========================================

PATTERN = r""""'(?:[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+| ?[^\s\p{L}\p{N}]+|\s+(?!\S)|\s+"""
PRETOKEN_REGEX = re.compile(PATTERN)

# ==========================================
# 2. 多进程预分词辅助函数 (保持原样, Python 的强项)
# ==========================================

def process_chunk_with_special_tokens(chunk_text: str, special_tokens: list[str], pretoken_regex: re.Pattern) -> Counter:
    split_pattern = "|".join(map(re.escape, special_tokens)) if special_tokens else "(?!)"
    text_segments = re.split(split_pattern, chunk_text)
    chunk_counts = Counter()
    for segment in text_segments:
        if not segment: continue
        for match in pretoken_regex.finditer(segment):
            pre_token = match.group()
            byte_token = pre_token.encode("utf-8")
            chunk_counts[byte_token] += 1
    return chunk_counts

def find_chunk_boundaries(file: BinaryIO, desired_num_chunks: int, split_special_token: bytes) -> list[int]:
    if not split_special_token:
        file.seek(0, os.SEEK_END)
        file_size = file.tell()
        chunk_size = file_size // desired_num_chunks
        boundaries = [i * chunk_size for i in range(desired_num_chunks + 1)]
        boundaries[-1] = file_size
        return sorted(set(boundaries))

    assert isinstance(split_special_token, bytes)
    file.seek(0, os.SEEK_END)
    file_size = file.tell()
    file.seek(0)
    chunk_size = file_size // desired_num_chunks
    chunk_boundaries = [i * chunk_size for i in range(desired_num_chunks + 1)]
    chunk_boundaries[-1] = file_size
    mini_chunk_size = 4096
    for bi in range(1, len(chunk_boundaries) - 1):
        initial_position = chunk_boundaries[bi]
        file.seek(initial_position)
        while True:
            mini_chunk = file.read(mini_chunk_size)
            if mini_chunk == b"":
```

```
                chunk_boundaries[bi] = file_size
                break
            found_at = mini_chunk.find(split_special_token)
            if found_at != -1:
                chunk_boundaries[bi] = initial_position + found_at
                break
            initial_position += mini_chunk_size
    return sorted(set(chunk_boundaries))

def worker_task(file_path: str, start: int, end: int, special_tokens: list[str]) -> Counter:
    with open(file_path, "rb") as f:
        f.seek(start)
        chunk_bytes = f.read(end - start)
    chunk_text = chunk_bytes.decode("utf-8", errors="ignore")
    return process_chunk_with_special_tokens(chunk_text, special_tokens, PRETOKEN_REGEX)

def run_parallel_pretokenization(input_path: str, special_tokens: list[str]) -> Counter:
    num_workers = multiprocessing.cpu_count()
    num_tasks = num_workers * 4
    boundary_token = special_tokens[0].encode('utf-8') if special_tokens else b""
    with open(input_path, "rb") as f:
        boundaries = find_chunk_boundaries(f, num_tasks, boundary_token)
    with multiprocessing.Pool(processes=num_workers) as pool:
        tasks = [(input_path, start, end, special_tokens) for start, end in zip(boundaries[:-1], boundaries[1:])]
        partial_counts = pool.starmap(worker_task, tasks)
    global_counts = Counter()
    for local_count in partial_counts:
        global_counts.update(local_count)
    return global_counts


# ==========================================
# 3. 核心大作业交付函数: train_bpe
# ==========================================

def train_bpe(input_path: str, vocab_size: int, special_tokens: list[str]) -> tuple[dict[int, bytes],
↪ list[tuple[bytes, bytes]]]:
    print(" 开始预分词与频率统计 (Python 多进程)...")
    global_counts = run_parallel_pretokenization(input_path, special_tokens)

    num_merges = vocab_size - 256 - len(special_tokens)
    if num_merges < 0:
        raise ValueError("vocab_size is too small to fit base bytes and special tokens.")

    print(f" 将数据移交 Rust 底层进行合并 (目标总词表大小: {vocab_size - len(special_tokens)})...")

    merges, vocab = fast_bpe.train_bpe_rust(global_counts, vocab_size - len(special_tokens))

    print("Rust 计算完成, 添加 Special Tokens...")

    vocab_idx = 256 + len(merges)
    for st in special_tokens:
        vocab[vocab_idx] = st.encode("utf-8")
        vocab_idx += 1

    print("BPE 训练彻底完成! ")
    return vocab, merges


```

```python
# ==========================================
# 测试/执行入口
# ==========================================
if __name__ == "__main__":

    file_path = "/home/inferior/Projects/CS336/data/owt_train.txt"
    # file_path = "/home/inferior/Projects/CS336/data/TinyStoriesV2-GPT4-train.txt"
    target_vocab_size = 32000
    # target_vocab_size = 10000
    special_tks = ["<|endoftext|>"]

    print("--- 开始 BPE 训练任务 ---")
    start_time = time.time()

    vocab, merges = train_bpe(
        input_path=file_path,
        vocab_size=target_vocab_size,
        special_tokens=special_tks
    )

    end_time = time.time()
    print(f"\n 训练总耗时: {end_time - start_time:.2f} 秒")
    print(f" 最终合并列表长度: {len(merges)}")
    print(f" 最终词表大小: {len(vocab)}")

    # ---------------------------------------------------------
    # 序列化保存 (Serialization)
    # ---------------------------------------------------------
    output_dir = "tokenizer_outputs"
    os.makedirs(output_dir, exist_ok=True)

    vocab_path = os.path.join(output_dir, "vocab.pkl")
    merges_path = os.path.join(output_dir, "merges.pkl")

    with open(vocab_path, "wb") as f:
        pickle.dump(vocab, f)

    with open(merges_path, "wb") as f:
        pickle.dump(merges, f)

    print(f" 序列化完成! 文件已保存至: {output_dir}/")

    # ---------------------------------------------------------
    # 分析: 寻找最长的 Token (作业要求)
    # ---------------------------------------------------------
    longest_token_id = max(vocab, key=lambda k: len(vocab[k]))
    longest_token_bytes = vocab[longest_token_id]

    print("\n 词表分析:")
    print(f" 最长 Token 的 ID: {longest_token_id}")
    print(f" 最长 Token 的字节长度: {len(longest_token_bytes)}")
    try:
        print(f" 最长 Token 的文本内容: {longest_token_bytes.decode('utf-8')!r}")
    except UnicodeDecodeError:
        print(f" 最长 Token (原始字节): {longest_token_bytes}")

    print("\n最初的 5 次 Merge:")
    for i in range(min(5, len(merges))):
        p1, p2 = merges[i]
```

```
        print(f"  {p1} + {p2} -> {p1 + p2}")
```

A.3 Tokenizer

```python
import pickle
import regex as re
from typing import Iterable, Iterator

# ==========================================
# 1. 全局配置与正则定义
# ==========================================
PATTERN = r"""'(?:[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+| ?[^\s\p{L}\p{N}]+|\s+(?!\S)|\s+"""
PRETOKEN_REGEX = re.compile(PATTERN)

# ==========================================
# 2. Tokenizer 核心类
# ==========================================
class Tokenizer:
    def __init__(self, vocab: dict[int, bytes], merges: list[tuple[bytes, bytes]], special_tokens: list[str] | None = None):
        """
        初始化 Tokenizer，建立查找索引并处理特殊 Token。
        """
        self.vocab = vocab.copy()
        self.merges = merges
        self.special_tokens = special_tokens or []

        # 建立反向查找字典，用于快速获取 token 对应的 ID
        self.token_to_id = {v: k for k, v in self.vocab.items()}

        # 将 merges 列表转化为 {pair: rank} 字典，以 O(1) 的时间复杂度获取合并优先级
        self.merge_ranks = {pair: i for i, pair in enumerate(self.merges)}

        # 动态处理传入的 special_tokens：如果不在词表中，则追加到末尾
        next_id = max(self.vocab.keys()) + 1 if self.vocab else 0
        for st in self.special_tokens:
            st_bytes = st.encode("utf-8")
            if st_bytes not in self.token_to_id:
                self.vocab[next_id] = st_bytes
                self.token_to_id[st_bytes] = next_id
                next_id += 1

        # 编译用于切分特殊 Token 的正则表达式
        self.st_pattern = None
        if self.special_tokens:
            # 使用捕获组 (...)，这样 re.split 时能保留特殊 token 在列表中
            sorted_st = sorted(self.special_tokens, key=len, reverse=True)
            pattern_str = "|".join(map(re.escape, sorted_st))
            self.st_pattern = re.compile(f"({pattern_str})")

    @classmethod
    def from_files(cls, vocab_filepath: str, merges_filepath: str, special_tokens: list[str] | None = None):
        """
        从序列化的 pickle 文件中加载 vocab 和 merges 并实例化 Tokenizer。
        """
        with open(vocab_filepath, "rb") as f:
            vocab = pickle.load(f)
        with open(merges_filepath, "rb") as f:
            merges = pickle.load(f)

        return cls(vocab, merges, special_tokens)
```

```python

    def _bpe_encode_chunk(self, chunk_text: str) -> list[int]:
        """
        内部辅助函数：对单个正则切割后的纯文本 pre-token 进行 BPE 合并操作。
        """
        # 将字符串打散成单字节列表
        b_list = [bytes([b]) for b in chunk_text.encode("utf-8")]

        # 如果长度为 1，直接返回（基础 256 字节必然在词表中）
        if len(b_list) <= 1:
            return [self.token_to_id[b] for b in b_list]

        while True:
            # 找到当前序列中，在 merges 中优先级最高（rank 最小）的相邻字节对
            min_rank = float('inf')
            best_pair = None

            for i in range(len(b_list) - 1):
                pair = (b_list[i], b_list[i+1])
                rank = self.merge_ranks.get(pair)
                if rank is not None and rank < min_rank:
                    min_rank = rank
                    best_pair = pair

            # 如果没有任何一对在 merge_ranks 中，说明合并彻底完成
            if best_pair is None:
                break

            # 执行最高优先级的一轮合并
            new_b_list = []
            i = 0
            while i < len(b_list):
                # 命中目标 pair
                if i < len(b_list) - 1 and b_list[i] == best_pair[0] and b_list[i+1] == best_pair[1]:
                    new_b_list.append(best_pair[0] + best_pair[1])
                    i += 2
                else:
                    new_b_list.append(b_list[i])
                    i += 1

            b_list = new_b_list

            # 如果只剩下一个完整的 token，提前退出
            if len(b_list) == 1:
                break

        # 拿着最终合并好的字节序列，去词表里换取对应的 IDs
        return [self.token_to_id[b] for b in b_list]

    def encode(self, text: str) -> list[int]:
        """
        将整段输入文本编码为 token IDs。
        """
        ids = []

        # 1. 全局截断：优先将文本通过 special_tokens 切开
        if self.st_pattern:
            parts = self.st_pattern.split(text)
        else:
```

```
            parts = [text]

        for part in parts:
            if not part:
                continue

            # 2. 如果这块 part 是一个 special token, 直接查 ID
            if part in self.special_tokens:
                ids.append(self.token_to_id[part.encode("utf-8")])
            else:
                # 3. 对普通文本进行预分词匹配
                for match in PRETOKEN_REGEX.finditer(part):
                    chunk_text = match.group()
                    ids.extend(self._bpe_encode_chunk(chunk_text))

        return ids

    def encode_iterable(self, iterable: Iterable[str]) -> Iterator[int]:
        """
        懒加载生成器: 逐块/逐行读取流数据并输出 IDs, 确保内存开销维持在常数级别。
        说明: 对于标准的文件句柄 (File Handle), 每次 yield 的是一行。
        换行符属于 BPE 空格家族, 因此按行拆分不会截断合法的词组边界。
        """
        for chunk in iterable:
            yield from self.encode(chunk)

    def decode(self, ids: list[int]) -> str:
        """
        将整数 ID 序列还原为字符串。非法字节将被自动替换为 U+FFFD ()。
        """
        raw_bytes = b"".join(self.vocab[idx] for idx in ids)
        return raw_bytes.decode("utf-8", errors="replace")
```

A.4 Adapters

```python
from __future__ import annotations

import os
import collections
import json
import regex
from collections.abc import Iterable
from typing import IO, Any, BinaryIO, Dict, List, Tuple

import numpy.typing as npt
import torch
from jaxtyping import Bool, Float, Int
from torch import Tensor
from collections import defaultdict
import pickle

from cs336_basics.test_function.bpe_trainer import train_bpe
from cs336_basics.test_function.tokenizer import Tokenizer
from cs336_basics.Chap_2.linear import Linear
from cs336_basics.Chap_2.embedding import Embedding
from cs336_basics.Chap_2.rmsnorm import RMSNorm
from cs336_basics.Chap_2.swiglu import SwiGLU

def run_linear(
    d_in: int,
    d_out: int,
    weights: Float[Tensor, " d_out d_in"],
    in_features: Float[Tensor, " ... d_in"],
) -> Float[Tensor, " ... d_out"]:
    """
    Given the weights of a Linear layer, compute the transformation of a batched input.

    Args:
        in_dim (int): The size of the input dimension
        out_dim (int): The size of the output dimension
        weights (Float[Tensor, "d_out d_in"]): The linear weights to use
        in_features (Float[Tensor, "... d_in"]): The output tensor to apply the function to

    Returns:
        Float[Tensor, "... d_out"]: The transformed output of your linear module.
    """

    model = Linear(
        in_features=d_in,
        out_features=d_out,
        device=in_features.device,
        dtype=in_features.dtype,
    )

    model.load_state_dict({"W": weights}, strict=True)

    model.eval()
    with torch.no_grad():
        output = model(in_features)

    return output

```

```python

def run_embedding(
    vocab_size: int,
    d_model: int,
    weights: Float[Tensor, " vocab_size d_model"],
    token_ids: Int[Tensor, " ..."],
) -> Float[Tensor, " ... d_model"]:
    """
    Given the weights of an Embedding layer, get the embeddings for a batch of token ids.

    Args:
        vocab_size (int): The number of embeddings in the vocabulary
        d_model (int): The size of the embedding dimension
        weights (Float[Tensor, "vocab_size d_model"]): The embedding vectors to fetch from
        token_ids (Int[Tensor, "..."]): The set of token ids to fetch from the Embedding layer

    Returns:
        Float[Tensor, "... d_model"]: Batch of embeddings returned by your Embedding layer.
    """

    model = Embedding(
        num_embeddings=vocab_size,
        embedding_dim=d_model,
        device=weights.device,
        dtype=weights.dtype
    )

    model.load_state_dict({"weight": weights}, strict=True)

    model.eval()
    with torch.no_grad():
        output = model(token_ids)

    return output


def run_swiglu(
    d_model: int,
    d_ff: int,
    w1_weight: Float[Tensor, " d_ff d_model"],
    w2_weight: Float[Tensor, " d_model d_ff"],
    w3_weight: Float[Tensor, " d_ff d_model"],
    in_features: Float[Tensor, " ... d_model"],
) -> Float[Tensor, " ... d_model"]:
    """Given the weights of a SwiGLU network, return
    the output of your implementation with these weights.

    Args:
        d_model (int): Dimensionality of the feedforward input and output.
        d_ff (int): Dimensionality of the up-project happening internally to your swiglu.
        w1_weight (Float[Tensor, "d_ff d_model"]): Stored weights for W1
        w2_weight (Float[Tensor, "d_model d_ff"]): Stored weights for W2
        w3_weight (Float[Tensor, "d_ff d_model"]): Stored weights for W3
        in_features (Float[Tensor, "... d_model"]): Input embeddings to the feed-forward layer.

    Returns:
        Float[Tensor, "... d_model"]: Output embeddings of the same shape as the input embeddings.
    """
    # Example:
```

```
    # If your state dict keys match, you can use `load_state_dict()`
    # swiglu.load_state_dict(weights)
    # You can also manually assign the weights
    # swiglu.w1.weight.data = w1_weight
    # swiglu.w2.weight.data = w2_weight
    # swiglu.w3.weight.data = w3_weight
    model = SwiGLU(
        d_model=d_model,
        d_ff=d_ff,
        device=in_features.device,
        dtype=in_features.dtype
    )

    state_dict = {
        "w1.W": w1_weight,
        "w2.W": w2_weight,
        "w3.W": w3_weight
    }
    model.load_state_dict(state_dict, strict=True)

    model.eval()
    with torch.no_grad():
        output = model(in_features)

    return output


def run_scaled_dot_product_attention(
    Q: Float[Tensor, " ... queries d_k"],
    K: Float[Tensor, " ... keys d_k"],
    V: Float[Tensor, " ... values d_v"],
    mask: Bool[Tensor, " ... queries keys"] | None = None,
) -> Float[Tensor, " ... queries d_v"]:
    """
    Given key (K), query (Q), and value (V) tensors, return
    the output of your scaled dot product attention implementation.

    Args:
        Q (Float[Tensor, " ... queries d_k"]): Query tensor
        K (Float[Tensor, " ... keys d_k"]): Key tensor
        V (Float[Tensor, " ... values d_v"]): Values tensor
        mask (Bool[Tensor, " ... queries keys"] | None): Mask tensor
    Returns:
        Float[Tensor, " ... queries d_v"]: Output of SDPA
    """
    raise NotImplementedError


def run_multihead_self_attention(
    d_model: int,
    num_heads: int,
    q_proj_weight: Float[Tensor, " d_k d_in"],
    k_proj_weight: Float[Tensor, " d_k d_in"],
    v_proj_weight: Float[Tensor, " d_v d_in"],
    o_proj_weight: Float[Tensor, " d_model d_v"],
    in_features: Float[Tensor, " ... sequence_length d_in"],
) -> Float[Tensor, " ... sequence_length d_out"]:
    """
    Given the key, query, and value projection weights of a naive unbatched
```

```
    implementation of multi-head attention, return the output of an optimized batched
    implementation. This implementation should handle the key, query, and value projections
    for all heads in a single matrix multiply.
    This function should not use RoPE.
    See section 3.2.2 of Vaswani et al., 2017.

    Args:
        d_model (int): Dimensionality of the feedforward input and output.
        num_heads (int): Number of heads to use in multi-headed attention.
        max_seq_len (int): Maximum sequence length to pre-cache if your implementation does that.
        q_proj_weight (Float[Tensor, "d_k d_in"]): Weights for the Q projection
        k_proj_weight (Float[Tensor, "d_k d_in"]): Weights for the K projection
        v_proj_weight (Float[Tensor, "d_k d_in"]): Weights for the V projection
        o_proj_weight (Float[Tensor, "d_model d_v"]): Weights for the output projection
        in_features (Float[Tensor, "... sequence_length d_in"]): Tensor to run your implementation on.

    Returns:
        Float[Tensor, " ... sequence_length d_out"]: Tensor with the output of running your optimized, batched multi-headed attention
        implementation with the given QKV projection weights and input features.
    """
    raise NotImplementedError


def run_multihead_self_attention_with_rope(
    d_model: int,
    num_heads: int,
    max_seq_len: int,
    theta: float,
    q_proj_weight: Float[Tensor, " d_k d_in"],
    k_proj_weight: Float[Tensor, " d_k d_in"],
    v_proj_weight: Float[Tensor, " d_v d_in"],
    o_proj_weight: Float[Tensor, " d_model d_v"],
    in_features: Float[Tensor, " ... sequence_length d_in"],
    token_positions: Int[Tensor, " ... sequence_length"] | None = None,
) -> Float[Tensor, " ... sequence_length d_out"]:
    """
    Given the key, query, and value projection weights of a naive unbatched
    implementation of multi-head attention, return the output of an optimized batched
    implementation. This implementation should handle the key, query, and value projections
    for all heads in a single matrix multiply.
    This version of MHA should include RoPE.
    In this case, the RoPE embedding dimension must be the head embedding dimension (d_model // num_heads).
    See section 3.2.2 of Vaswani et al., 2017.

    Args:
        d_model (int): Dimensionality of the feedforward input and output.
        num_heads (int): Number of heads to use in multi-headed attention.
        max_seq_len (int): Maximum sequence length to pre-cache if your implementation does that.
        theta (float): RoPE parameter.
        q_proj_weight (Float[Tensor, "d_k d_in"]): Weights for the Q projection
        k_proj_weight (Float[Tensor, "d_k d_in"]): Weights for the K projection
        v_proj_weight (Float[Tensor, "d_k d_in"]): Weights for the V projection
        o_proj_weight (Float[Tensor, "d_model d_v"]): Weights for the output projection
        in_features (Float[Tensor, "... sequence_length d_in"]): Tensor to run your implementation on.
        token_positions (Int[Tensor, " ... sequence_length"] | None): Optional tensor with the positions of the tokens

    Returns:
```

```
        Float[Tensor, " ... sequence_length d_out"]: Tensor with the output of running your optimized, batched multi-headed attention
        implementation with the given QKV projection weights and input features.
    """
    raise NotImplementedError


def run_rope(
    d_k: int,
    theta: float,
    max_seq_len: int,
    in_query_or_key: Float[Tensor, " ... sequence_length d_k"],
    token_positions: Int[Tensor, " ... sequence_length"],
) -> Float[Tensor, " ... sequence_length d_k"]:
    """
    Run RoPE for a given input tensor.

    Args:
        d_k (int): Embedding dimension size for the query or key tensor.
        theta (float): RoPE parameter.
        max_seq_len (int): Maximum sequence length to pre-cache if your implementation does that.
        in_query_or_key (Float[Tensor, "... sequence_length d_k"]): Input tensor to run RoPE on.
        token_positions (Int[Tensor, "... sequence_length"]): Tensor of shape (batch_size, sequence_length) with the token positions
    Returns:
        Float[Tensor, " ... sequence_length d_k"]: Tensor with RoPEd input.
    """
    raise NotImplementedError


def run_transformer_block(
    d_model: int,
    num_heads: int,
    d_ff: int,
    max_seq_len: int,
    theta: float,
    weights: dict[str, Tensor],
    in_features: Float[Tensor, " batch sequence_length d_model"],
) -> Float[Tensor, " batch sequence_length d_model"]:
    """
    Given the weights of a pre-norm Transformer block and input features,
    return the output of running the Transformer block on the input features.

    This function should use RoPE.
    Depending on your implementation, you may simply need to pass the relevant args
    to your TransformerBlock constructor, or you may need to initialize your own RoPE
    class and pass that instead.

    Args:
        d_model (int): The dimensionality of the Transformer block input.
        num_heads (int): Number of heads to use in multi-headed attention. `d_model` must be
            evenly divisible by `num_heads`.
        d_ff (int): Dimensionality of the feed-forward inner layer.
        max_seq_len (int): Maximum sequence length to pre-cache if your implementation does that.
        theta (float): RoPE parameter.
        weights (dict[str, Tensor]):
            State dict of our reference implementation.
            The keys of this dictionary are:
            - `attn.q_proj.weight`
```

```
                The query projections for all `num_heads` attention heads.
                Shape is (d_model, d_model).
                The rows are ordered by matrices of shape (num_heads, d_k),
                so `attn.q_proj.weight == torch.cat([q_heads.0.weight, ..., q_heads.N.weight], dim=0)`.
            - `attn.k_proj.weight`
                The key projections for all `num_heads` attention heads.
                Shape is (d_model, d_model).
                The rows are ordered by matrices of shape (num_heads, d_k),
                so `attn.k_proj.weight == torch.cat([k_heads.0.weight, ..., k_heads.N.weight], dim=0)`.
            - `attn.v_proj.weight`
                The value projections for all `num_heads` attention heads.
                Shape is (d_model, d_model).
                The rows are ordered by matrices of shape (num_heads, d_v),
                so `attn.v_proj.weight == torch.cat([v_heads.0.weight, ..., v_heads.N.weight], dim=0)`.
            - `attn.output_proj.weight`
                Weight of the multi-head self-attention output projection
                Shape is (d_model, d_model).
            - `ln1.weight`
                Weights of affine transform for the first RMSNorm
                applied in the transformer block.
                Shape is (d_model,).
            - `ffn.w1.weight`
                Weight of the first linear transformation in the FFN.
                Shape is (d_model, d_ff).
            - `ffn.w2.weight`
                Weight of the second linear transformation in the FFN.
                Shape is (d_ff, d_model).
            - `ffn.w3.weight`
                Weight of the third linear transformation in the FFN.
                Shape is (d_model, d_ff).
            - `ln2.weight`
                Weights of affine transform for the second RMSNorm
                applied in the transformer block.
                Shape is (d_model,).
        in_features (Float[Tensor, "batch sequence_length d_model"]):
            Tensor to run your implementation on.

    Returns:
        Float[Tensor, "batch sequence_length d_model"] Tensor with the output of
        running the Transformer block on the input features while using RoPE.
    """
    raise NotImplementedError


def run_transformer_lm(
    vocab_size: int,
    context_length: int,
    d_model: int,
    num_layers: int,
    num_heads: int,
    d_ff: int,
    rope_theta: float,
    weights: dict[str, Tensor],
    in_indices: Int[Tensor, " batch_size sequence_length"],
) -> Float[Tensor, " batch_size sequence_length vocab_size"]:
    r"""Given the weights of a Transformer language model and input indices,
    return the output of running a forward pass on the input indices.

    This function should use RoPE.
```

```

    Args:
        vocab_size (int): The number of unique items in the output vocabulary to be predicted.
        context_length (int): The maximum number of tokens to process at once.
        d_model (int): The dimensionality of the model embeddings and sublayer outputs.
        num_layers (int): The number of Transformer layers to use.
        num_heads (int): Number of heads to use in multi-headed attention. `d_model` must be
            evenly divisible by `num_heads`.
        d_ff (int): Dimensionality of the feed-forward inner layer (section 3.3).
        rope_theta (float): The RoPE $\Theta$ parameter.
        weights (dict[str, Tensor]):
            State dict of our reference implementation. {num_layers} refers to an
            integer between `0` and `num_layers - 1` (the layer index).
            The keys of this dictionary are:
            - `token_embeddings.weight`
                Token embedding matrix. Shape is (vocab_size, d_model).
            - `layers.{num_layers}.attn.q_proj.weight`
                The query projections for all `num_heads` attention heads.
                Shape is (num_heads * (d_model / num_heads), d_model).
                The rows are ordered by matrices of shape (num_heads, d_k),
                so `attn.q_proj.weight == torch.cat([q_heads.0.weight, ..., q_heads.N.weight], dim=0)`.
            - `layers.{num_layers}.attn.k_proj.weight`
                The key projections for all `num_heads` attention heads.
                Shape is (num_heads * (d_model / num_heads), d_model).
                The rows are ordered by matrices of shape (num_heads, d_k),
                so `attn.k_proj.weight == torch.cat([k_heads.0.weight, ..., k_heads.N.weight], dim=0)`.
            - `layers.{num_layers}.attn.v_proj.weight`
                The value projections for all `num_heads` attention heads.
                Shape is (num_heads * (d_model / num_heads), d_model).
                The rows are ordered by matrices of shape (num_heads, d_v),
                so `attn.v_proj.weight == torch.cat([v_heads.0.weight, ..., v_heads.N.weight], dim=0)`.
            - `layers.{num_layers}.attn.output_proj.weight`
                Weight of the multi-head self-attention output projection
                Shape is ((d_model / num_heads) * num_heads, d_model).
            - `layers.{num_layers}.ln1.weight`
                Weights of affine transform for the first RMSNorm
                applied in the transformer block.
                Shape is (d_model,).
            - `layers.{num_layers}.ffn.w1.weight`
                Weight of the first linear transformation in the FFN.
                Shape is (d_model, d_ff).
            - `layers.{num_layers}.ffn.w2.weight`
                Weight of the second linear transformation in the FFN.
                Shape is (d_ff, d_model).
            - `layers.{num_layers}.ffn.w3.weight`
                Weight of the third linear transformation in the FFN.
                Shape is (d_model, d_ff).
            - `layers.{num_layers}.ln2.weight`
                Weights of affine transform for the second RMSNorm
                applied in the transformer block.
                Shape is (d_model,).
            - `ln_final.weight`
                Weights of affine transform for RMSNorm applied to the output of the final transformer block.
                Shape is (d_model, ).
            - `lm_head.weight`
                Weights of the language model output embedding.
                Shape is (vocab_size, d_model).
        in_indices (Int[Tensor, "batch_size sequence_length"]) Tensor with input indices to run the language model
        ↪  on. Shape is (batch_size, sequence_length), where
```

```
            `sequence_length` is at most `context_length`.

    Returns:
        Float[Tensor, "batch_size sequence_length vocab_size"]: Tensor with the predicted unnormalized
        next-word distribution for each token.
    """
    raise NotImplementedError


def run_rmsnorm(
    d_model: int,
    eps: float,
    weights: Float[Tensor, " d_model"],
    in_features: Float[Tensor, " ... d_model"],
) -> Float[Tensor, " ... d_model"]:
    """Given the weights of a RMSNorm affine transform,
    return the output of running RMSNorm on the input features.

    Args:
        d_model (int): The dimensionality of the RMSNorm input.
        eps: (float): A value added to the denominator for numerical stability.
        weights (Float[Tensor, "d_model"]): RMSNorm weights.
        in_features (Float[Tensor, "... d_model"]): Input features to run RMSNorm on. Can have arbitrary leading
            dimensions.

    Returns:
        Float[Tensor,"... d_model"]: Tensor of with the same shape as `in_features` with the output of running
        RMSNorm of the `in_features`.
    """
    model = RMSNorm(
        d_model=d_model,
        eps=eps,
        device=weights.device,
        dtype=weights.dtype
    )

    model.load_state_dict({"weight": weights}, strict=True)

    model.eval()
    with torch.no_grad():
        output = model(in_features)

    return output


def run_silu(in_features: Float[Tensor, " ..."]) -> Float[Tensor, " ..."]:
    """Given a tensor of inputs, return the output of applying SiLU
    to each element.

    Args:
        in_features(Float[Tensor, "..."]): Input features to run SiLU on. Shape is arbitrary.

    Returns:
        Float[Tensor,"..."]: of with the same shape as `in_features` with the output of applying
        SiLU to each element.
    """
    raise NotImplementedError
```

```python
def run_get_batch(
    dataset: npt.NDArray, batch_size: int, context_length: int, device: str
) -> tuple[torch.Tensor, torch.Tensor]:
    """
    Given a dataset (a 1D numpy array of integers) and a desired batch size and
    context length, sample language modeling input sequences and their corresponding
    labels from the dataset.

    Args:
        dataset (np.array): 1D numpy array of integer token IDs in the dataset.
        batch_size (int): Desired batch size to sample.
        context_length (int): Desired context length of each sampled example.
        device (str): PyTorch device string (e.g., 'cpu' or 'cuda:0') indicating the device
            to place the sampled input sequences and labels on.

    Returns:
        Tuple of torch.LongTensors of shape (batch_size, context_length). The first tuple item
        is the sampled input sequences, and the second tuple item is the corresponding
        language modeling labels.
    """
    raise NotImplementedError


def run_softmax(in_features: Float[Tensor, " ..."], dim: int) -> Float[Tensor, " ..."]:
    """
    Given a tensor of inputs, return the output of softmaxing the given `dim`
    of the input.

    Args:
        in_features (Float[Tensor, "..."]): Input features to softmax. Shape is arbitrary.
        dim (int): Dimension of the `in_features` to apply softmax to.

    Returns:
        Float[Tensor, "..."]: Tensor of with the same shape as `in_features` with the output of
        softmax normalizing the specified `dim`.
    """
    raise NotImplementedError


def run_cross_entropy(
    inputs: Float[Tensor, " batch_size vocab_size"], targets: Int[Tensor, " batch_size"]
) -> Float[Tensor, ""]:
    """Given a tensor of inputs and targets, compute the average cross-entropy
    loss across examples.

    Args:
        inputs (Float[Tensor, "batch_size vocab_size"]): inputs[i][j] is the
            unnormalized logit of jth class for the ith example.
        targets (Int[Tensor, "batch_size"]): Tensor of shape (batch_size,) with the index of the correct class.
            Each value must be between 0 and `num_classes - 1`.

    Returns:
        Float[Tensor, ""]: The average cross-entropy loss across examples.
    """
    raise NotImplementedError


def run_gradient_clipping(parameters: Iterable[torch.nn.Parameter], max_l2_norm: float) -> None:
    """Given a set of parameters, clip their combined gradients to have l2 norm at most max_l2_norm.
```

```python

    Args:
        parameters (Iterable[torch.nn.Parameter]): collection of trainable parameters.
        max_l2_norm (float): a positive value containing the maximum l2-norm.

    The gradients of the parameters (parameter.grad) should be modified in-place.
    """
    raise NotImplementedError


def get_adamw_cls() -> Any:
    """
    Returns a torch.optim.Optimizer that implements AdamW.
    """
    raise NotImplementedError


def run_get_lr_cosine_schedule(
    it: int,
    max_learning_rate: float,
    min_learning_rate: float,
    warmup_iters: int,
    cosine_cycle_iters: int,
):
    """
    Given the parameters of a cosine learning rate decay schedule (with linear
    warmup) and an iteration number, return the learning rate at the given
    iteration under the specified schedule.

    Args:
        it (int): Iteration number to get learning rate for.
        max_learning_rate (float): alpha_max, the maximum learning rate for
            cosine learning rate schedule (with warmup).
        min_learning_rate (float): alpha_min, the minimum / final learning rate for
            the cosine learning rate schedule (with warmup).
        warmup_iters (int): T_w, the number of iterations to linearly warm-up
            the learning rate.
        cosine_cycle_iters (int): T_c, the number of cosine annealing iterations.

    Returns:
        Learning rate at the given iteration under the specified schedule.
    """
    raise NotImplementedError


def run_save_checkpoint(
    model: torch.nn.Module,
    optimizer: torch.optim.Optimizer,
    iteration: int,
    out: str | os.PathLike | BinaryIO | IO[bytes],
):
    """
    Given a model, optimizer, and an iteration number, serialize them to disk.

    Args:
        model (torch.nn.Module): Serialize the state of this model.
        optimizer (torch.optim.Optimizer): Serialize the state of this optimizer.
        iteration (int): Serialize this value, which represents the number of training iterations
            we've completed.
```

```
        out (str | os.PathLike | BinaryIO | IO[bytes]): Path or file-like object to serialize the model, optimizer, and iteration to.
    """
    raise NotImplementedError


def run_load_checkpoint(
    src: str | os.PathLike | BinaryIO | IO[bytes],
    model: torch.nn.Module,
    optimizer: torch.optim.Optimizer,
) -> int:
    """
    Given a serialized checkpoint (path or file-like object), restore the
    serialized state to the given model and optimizer.
    Return the number of iterations that we previously serialized in
    the checkpoint.

    Args:
        src (str | os.PathLike | BinaryIO | IO[bytes]): Path or file-like object to serialized checkpoint.
        model (torch.nn.Module): Restore the state of this model.
        optimizer (torch.optim.Optimizer): Restore the state of this optimizer.
    Returns:
        int: the previously-serialized number of iterations.
    """
    raise NotImplementedError


def get_tokenizer(
    vocab: dict[int, bytes],
    merges: list[tuple[bytes, bytes]],
    special_tokens: list[str] | None = None,
) -> Any:
    """Given a vocabulary, a list of merges, and a list of special tokens,
    return a BPE tokenizer that uses the provided vocab, merges, and special tokens.

    Args:
        vocab (dict[int, bytes]): The tokenizer vocabulary, a mapping from int (token ID in the vocabulary)
            to bytes (token bytes)
        merges (list[tuple[bytes, bytes]]): BPE merges. Each list item is a tuple of bytes (<token1>, <token2>),
            representing that <token1> was merged with <token2>.
            Merges are ordered by order of creation.
        special_tokens (list[str] | None): A list of string special tokens for the tokenizer. These strings will never
            be split into multiple tokens, and will always be kept as a single token.

    Returns:
        A BPE tokenizer that uses the provided vocab, merges, and special tokens.
    """
    return Tokenizer(vocab=vocab, merges=merges, special_tokens=special_tokens)


def run_train_bpe(
    input_path: str,
    vocab_size: int,
    special_tokens: list[str]
) -> tuple[dict[int, bytes], list[tuple[bytes, bytes]]]:
    """
    Adapter function for training the BPE tokenizer.
    This routes the inputs from the test suite to our optimized implementation.
```

```
    """

    # 直接调用核心逻辑并返回结果
    vocab, merges = train_bpe(
        input_path=input_path,
        vocab_size=vocab_size,
        special_tokens=special_tokens
    )

    return vocab, merges
```